

Técnicas de Diseño de Algoritmos (Ex Algoritmos y
Estructuras de Datos III)

Segundo cuatrimestre 2025

Problema de flujo máximo

Flujo en redes

► Datos de entrada:

1. Un grafo dirigido $G = (N, A)$.
2. Nodos $s, t \in N$ de origen y destino.
3. Una función de **capacidad** $u : A \rightarrow \mathbb{Z}_+$ asociada con los arcos.

Flujo en redes

► Datos de entrada:

1. Un grafo dirigido $G = (N, A)$.
2. Nodos $s, t \in N$ de origen y destino.
3. Una función de **capacidad** $u : A \rightarrow \mathbb{Z}_+$ asociada con los arcos.

- **Problema:** Encontrar un **flujo** (cantidad a enviar por cada arco) entre s y t de mayor valor posible.

Flujo en redes

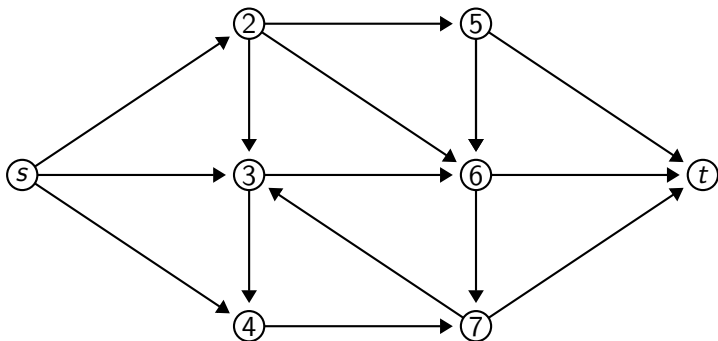
► Datos de entrada:

1. Un grafo dirigido $G = (N, A)$.
2. Nodos $s, t \in N$ de origen y destino.
3. Una función de **capacidad** $u : A \rightarrow \mathbb{Z}_+$ asociada con los arcos.

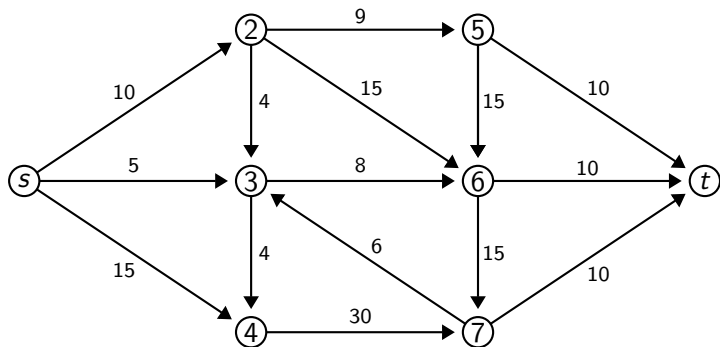
► **Problema:** Encontrar un **flujo** (cantidad a enviar por cada arco) entre s y t de mayor valor posible.

1. Salvo s y t , en cada nodo la cantidad de flujo que entra al nodo debe ser igual a la cantidad de flujo que sale del nodo.
2. La cantidad x_{ij} enviada por el arco $ij \in A$ debe cumplir $0 \leq x_{ij} \leq u_{ij}$.
3. El **valor** de un flujo es la cantidad de **flujo neto** que sale de s .

Flujo en redes

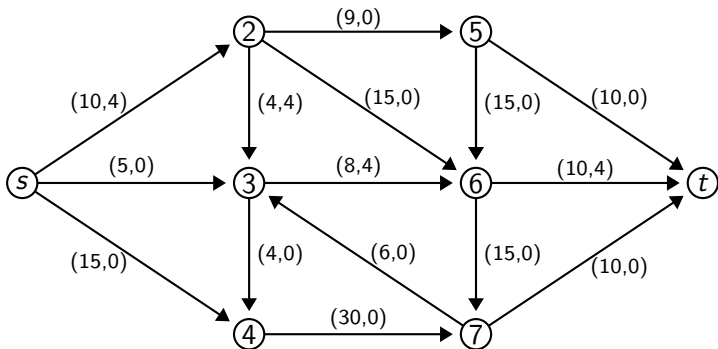


Flujo en redes



Flujo en redes

$$F = 4$$



Flujo en redes

- Un **corte** en la red $G = (N, A)$ es un subconjunto $S \subseteq N \setminus \{t\}$ tal que $s \in S$.

Flujo en redes

- ▶ Un **corte** en la red $G = (N, A)$ es un subconjunto $S \subseteq N \setminus \{t\}$ tal que $s \in S$.
- ▶ Dados $S, T \subseteq N$, definimos $ST = \{ij \in A : i \in S \text{ y } j \in T\}$

Flujo en redes

- ▶ Un **corte** en la red $G = (N, A)$ es un subconjunto $S \subseteq N \setminus \{t\}$ tal que $s \in S$.
- ▶ Dados $S, T \subseteq N$, definimos $ST = \{ij \in A : i \in S \text{ y } j \in T\}$
- ▶ **Proposición:** Sea x un flujo definido en una red $G = (N, A)$ y sea S un corte. Entonces

$$F = \sum_{ij \in S\bar{S}} x_{ij} - \sum_{ij \in \bar{S}S} x_{ij}$$

donde $\bar{S} = N \setminus S$.

Prueba de la Proposición

Claramente, vale para $S = \{s\}$. Es decir que

$$F = \sum_{ij \in \{s\}(N \setminus \{s\})} x_{ij} - \sum_{ij \in (N \setminus \{s\})\{s\}} x_{ij}$$

Sea ahora S cualquier corte, debemos probar que

$$\sum_{ij \in \{s\}(N \setminus \{s\})} x_{ij} - \sum_{ij \in (N \setminus \{s\})\{s\}} x_{ij} = \sum_{ij \in S\bar{S}} x_{ij} - \sum_{ij \in \bar{S}S} x_{ij}$$

Tomemos $S' = S \setminus \{s\}$ y claramente N es la unión disjunta de $\{s\}$, S' y $\bar{S}$.

$$\begin{aligned} \sum_{ij \in \{s\}S'} x_{ij} + \sum_{ij \in \{s\}\bar{S}} x_{ij} - \sum_{ij \in S'\{s\}} x_{ij} - \sum_{ij \in \bar{S}\{s\}} x_{ij} = \\ \sum_{ij \in \{s\}\bar{S}} x_{ij} + \sum_{ij \in S'\bar{S}} x_{ij} - \sum_{ij \in \bar{S}\{s\}} x_{ij} - \sum_{ij \in \bar{S}S'} x_{ij} \end{aligned}$$

Prueba de la Proposición: continuación

Equivale a la siguiente igualdad

$$\sum_{ij \in \{s\}S'} x_{ij} + \sum_{ij \in \bar{S}S'} x_{ij} = \sum_{ij \in S'\bar{S}} x_{ij} + \sum_{ij \in S'\{s\}} x_{ij}$$

Sumemos ahora

$$\sum_{ij \in S'S'} x_{ij}$$

a ambos lados de la igualdad.

$$\sum_{ij \in S'S'} x_{ij} + \sum_{ij \in \{s\}S'} x_{ij} + \sum_{ij \in \bar{S}S'} x_{ij} = \sum_{ij \in S'S'} x_{ij} + \sum_{ij \in S'\bar{S}} x_{ij} + \sum_{ij \in S'\{s\}} x_{ij}$$

Quedaría

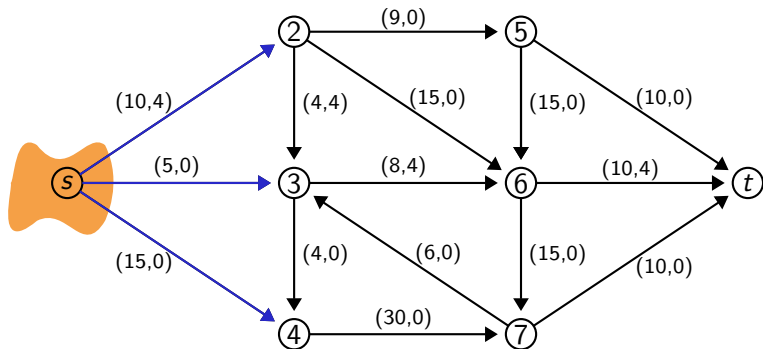
$$\sum_{ij \in NS'} x_{ij} = \sum_{ij \in S'N} x_{ij}$$

La cual es válida ya que $\forall k \in S'$ se cumple la ley de conservación,

$$\sum_{ij \in N\{k\}} x_{ij} = \sum_{ij \in \{k\}N} x_{ij}$$

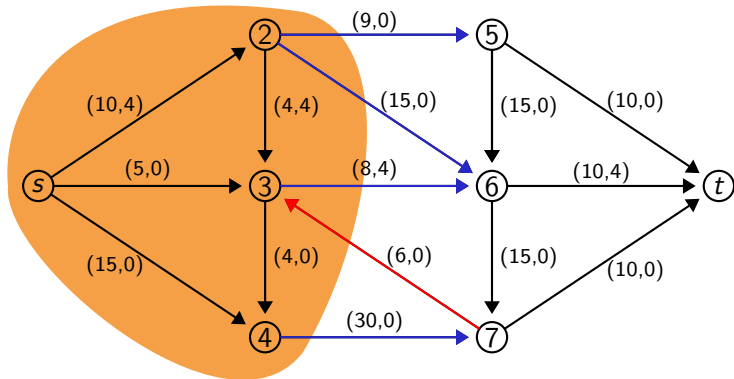
Flujo en redes

$$F = 4$$



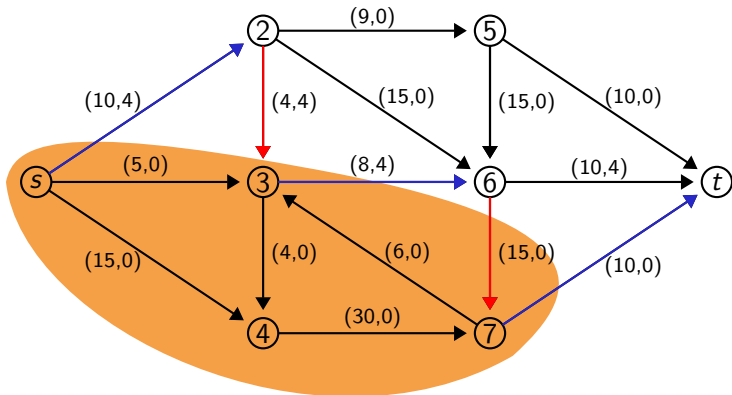
Flujo en redes

$$F = 4$$



Flujo en redes

$$F = 4$$



Flujo en redes

- ▶ La **capacidad** de un corte S se define como

$$u(S) = \sum_{ij \in S\bar{S}} u_{ij}.$$

Flujo en redes

- ▶ La **capacidad** de un corte S se define como

$$u(S) = \sum_{ij \in S\bar{S}} u_{ij}.$$

- ▶ **Proposición:** Si x es un flujo con valor F y S es un corte en N , entonces $F \leq u(S)$.

Flujo en redes

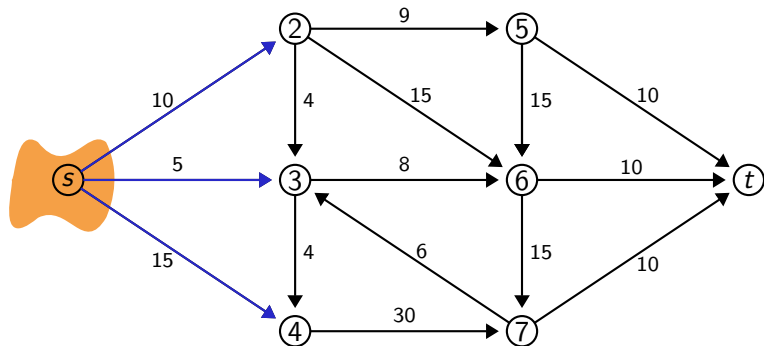
- ▶ La **capacidad** de un corte S se define como

$$u(S) = \sum_{ij \in S\bar{S}} u_{ij}.$$

- ▶ **Proposición:** Si x es un flujo con valor F y S es un corte en N , entonces $F \leq u(S)$.
- ▶ **Corolario (certificado de optimalidad):** Si F es el valor de un flujo x y S un corte en G tal que $F = u(S)$ entonces x define un flujo máximo y S un corte de capacidad mínima.

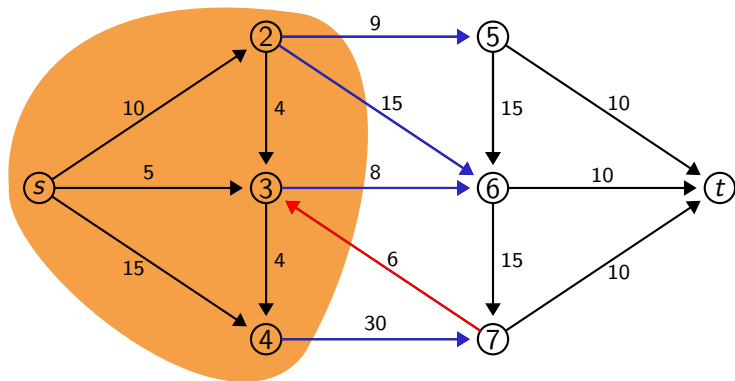
Flujo en redes

$$U = 30$$



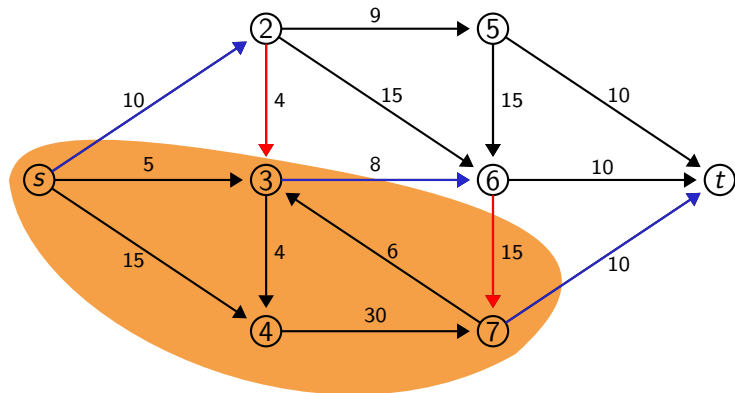
Flujo en redes

$$U = 62$$



Flujo en redes

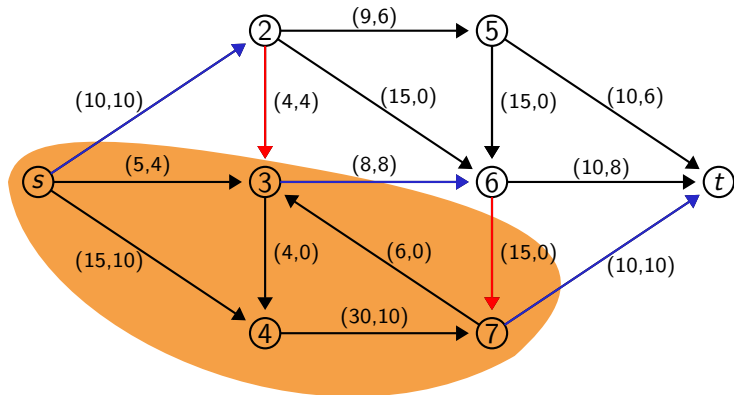
$$U = 28$$



Flujo en redes

$$U = 28$$

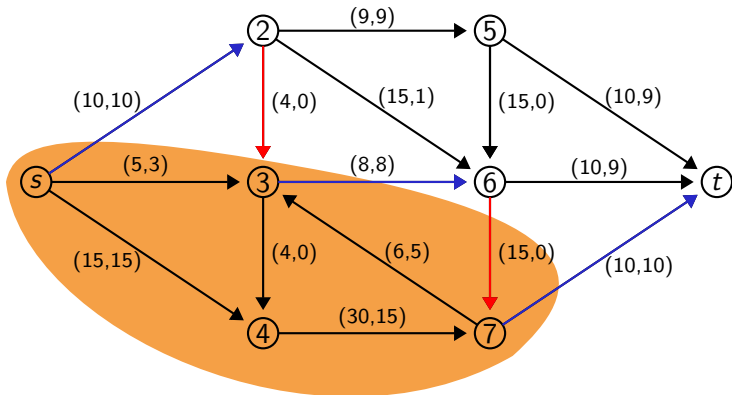
$$F = 24$$



Flujo en redes

$$U = 28$$

$$F = 28$$

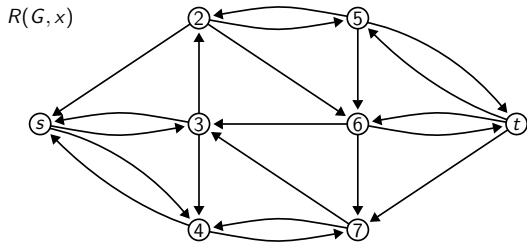
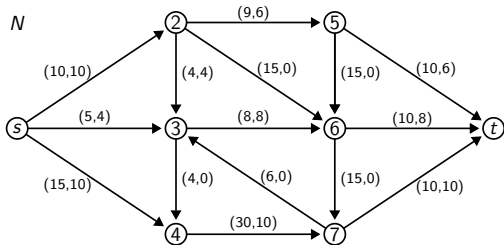


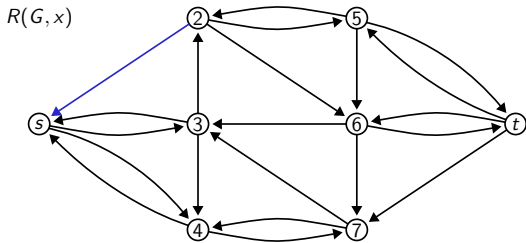
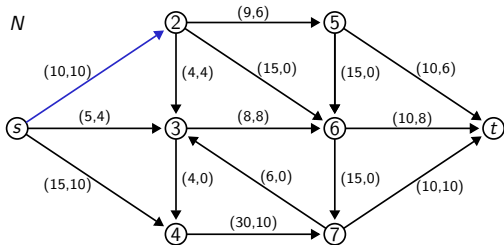
Flujo en redes - Camino de aumento

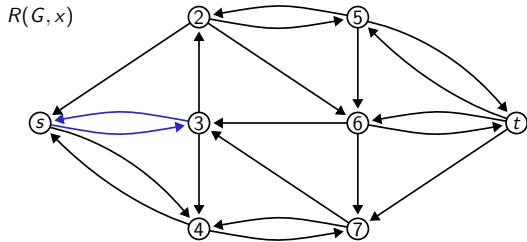
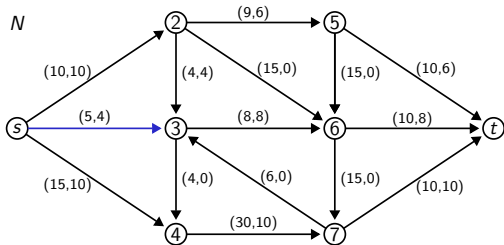
- Dada una red $G = (N, A)$ con función de capacidad u y un flujo factible x , definimos la **red residual** $R(G, x) = (N, A_R)$, donde:
 1. $ij \in A_R$ si $x_{ij} < u_{ij}$,
 2. $ji \in A_R$ si $x_{ij} > 0$.

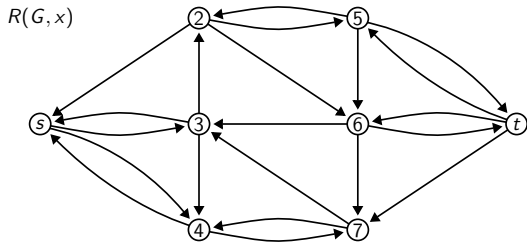
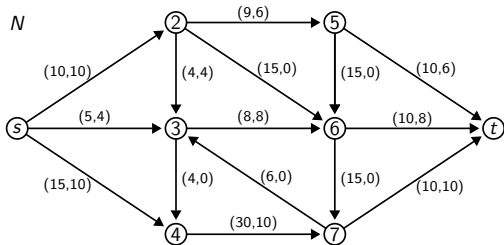
Flujo en redes - Camino de aumento

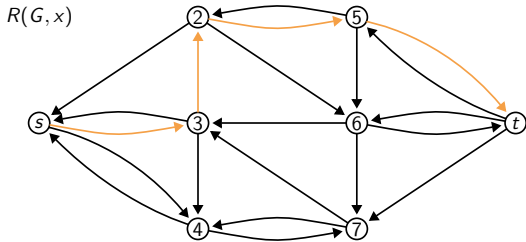
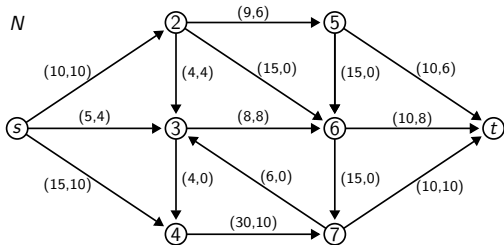
- ▶ Dada una red $G = (N, A)$ con función de capacidad u y un flujo factible x , definimos la **red residual** $R(G, x) = (N, A_R)$, donde:
 1. $ij \in A_R$ si $x_{ij} < u_{ij}$,
 2. $ji \in A_R$ si $x_{ij} > 0$.
- ▶ Un **camino de aumento** es un camino orientado de s a t en $R(G, x)$.











Flujo en redes - Camino de aumento

- Dado un camino de aumento P , para cada arco $ij \in P$ definimos

$$\Delta(ij) = \begin{cases} u_{ij} - x_{ij} & \text{si } ij \in A \\ x_{ji} & \text{si } ji \in A \end{cases}$$

Flujo en redes - Camino de aumento

- ▶ Dado un camino de aumento P , para cada arco $ij \in P$ definimos

$$\Delta(ij) = \begin{cases} u_{ij} - x_{ij} & \text{si } ij \in A \\ x_{ji} & \text{si } ji \in A \end{cases}$$

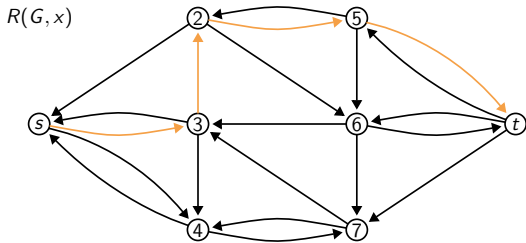
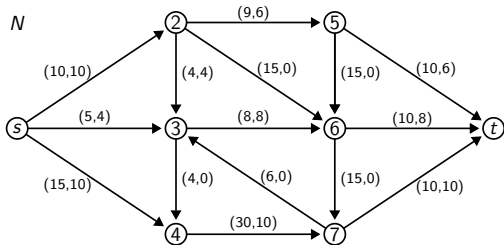
- ▶ Definimos además $\Delta(P) = \min_{ij \in P} \{\Delta(ij)\}$.

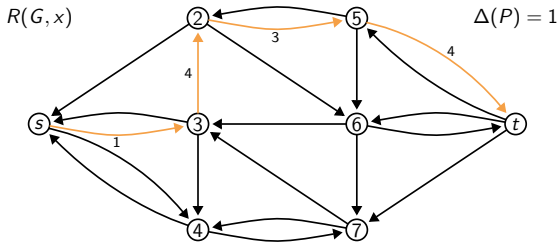
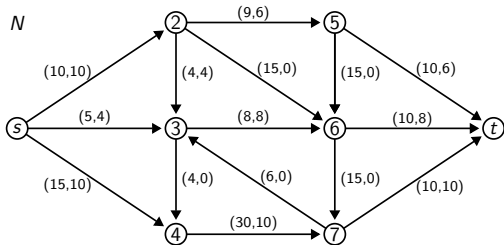
Flujo en redes - Camino de aumento

- ▶ Dado un camino de aumento P , para cada arco $ij \in P$ definimos

$$\Delta(ij) = \begin{cases} u_{ij} - x_{ij} & \text{si } ij \in A \\ x_{ji} & \text{si } ji \in A \end{cases}$$

- ▶ Definimos además $\Delta(P) = \min_{ij \in P} \{\Delta(ij)\}$.
- ▶ Podemos encontrar un camino de aumento P en la red residual en $O(m)$, y calculamos $\Delta(P)$ en $O(n)$.





Flujo en redes

- **Proposición:** Sea x un flujo definido sobre una red N con valor F y sea P un camino de aumento en $R(G, x)$. Entonces el flujo $\bar{x}$, definido por

$$\bar{x}(ij) = \begin{cases} x_{ij} & \text{si } ij \notin P \wedge ji \notin P \\ x_{ij} + \Delta(P) & \text{si } ij \in P \\ x_{ij} - \Delta(P) & \text{si } ji \in P \end{cases}$$

es un flujo factible sobre N con valor $\bar{F} = F + \Delta(P)$.

Flujo en redes

- **Proposición:** Sea x un flujo definido sobre una red N con valor F y sea P un camino de aumento en $R(G, x)$. Entonces el flujo $\bar{x}$, definido por

$$\bar{x}(ij) = \begin{cases} x_{ij} & \text{si } ij \notin P \wedge ji \notin P \\ x_{ij} + \Delta(P) & \text{si } ij \in P \\ x_{ij} - \Delta(P) & \text{si } ji \in P \end{cases}$$

es un flujo factible sobre N con valor $\bar{F} = F + \Delta(P)$.

- **Teorema:** Sea x un flujo definido sobre una red N . Entonces x es un flujo máximo $\iff$ no existe camino de aumento en $R(G, x)$.

Flujo en redes

- **Proposición:** Sea x un flujo definido sobre una red N con valor F y sea P un camino de aumento en $R(G, x)$. Entonces el flujo $\bar{x}$, definido por

$$\bar{x}(ij) = \begin{cases} x_{ij} & \text{si } ij \notin P \wedge ji \notin P \\ x_{ij} + \Delta(P) & \text{si } ij \in P \\ x_{ij} - \Delta(P) & \text{si } ji \in P \end{cases}$$

es un flujo factible sobre N con valor $\bar{F} = F + \Delta(P)$.

- **Teorema:** Sea x un flujo definido sobre una red N . Entonces x es un flujo máximo $\iff$ no existe camino de aumento en $R(G, x)$.
- **Teorema (max flow-min cut):** Dada una red N , el valor del **flujo máximo** es igual a la capacidad del **corte mínimo**.

Algoritmo de Ford y Fulkerson



Lester Ford
(1927–2017)



Delbert Fulkerson
(1924–1976)

- ▶ El algoritmo de Ford y Fulkerson (1956) obtiene un flujo máximo con complejidad $O(nmU)$, donde $U = \max_{ij \in A} u_{ij}$.

Algoritmo de Ford y Fulkerson

Definir un flujo inicial en N (por ejemplo, $x = 0$)

mientras exista $P :=$ camino de aumento en $R(G, x)$ **hacer**

para cada arco $ij \in P$ **hacer**

si $ij \in A$ **entonces**

$x_{ij} := x_{ij} + \Delta(P)$

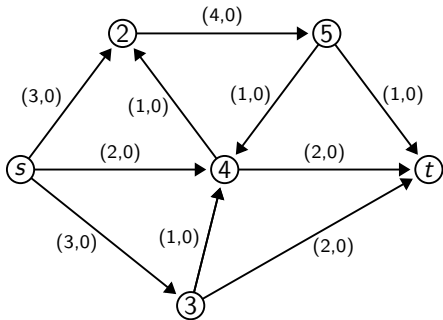
si no ($ji \in A$)

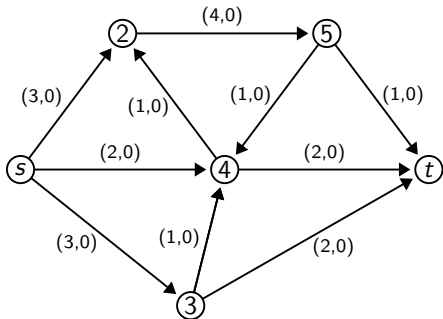
$x_{ji} := x_{ji} - \Delta(P)$

fin si

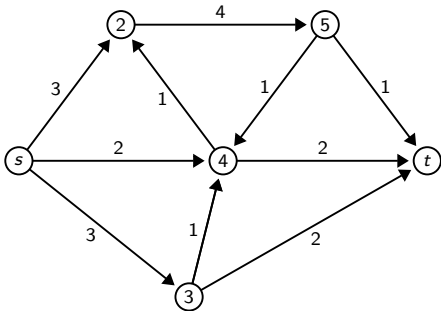
fin para

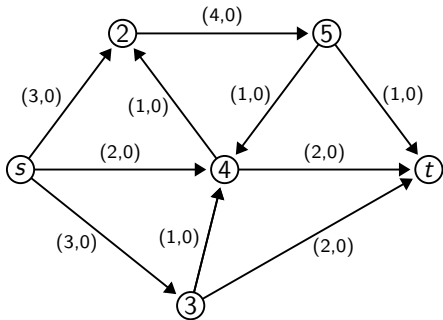
fin mientras



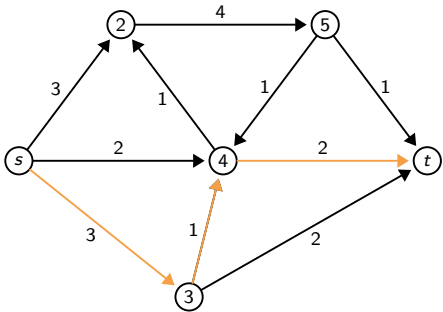


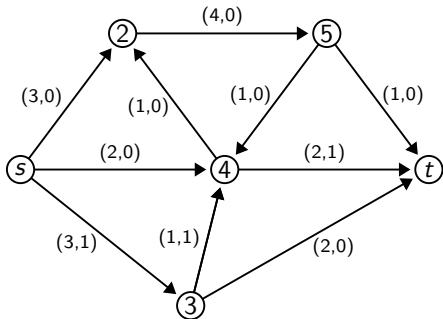
$R(G, x)$



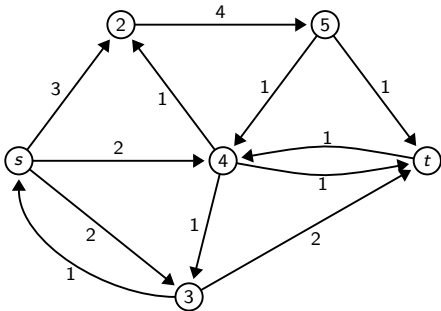


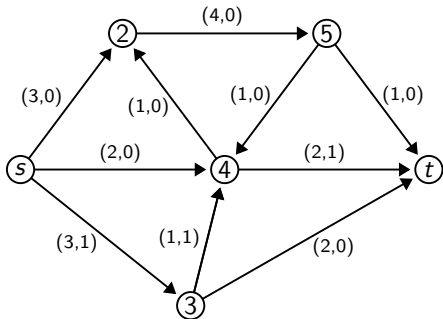
$R(G, x)$



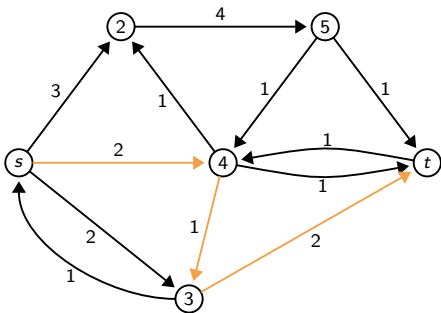


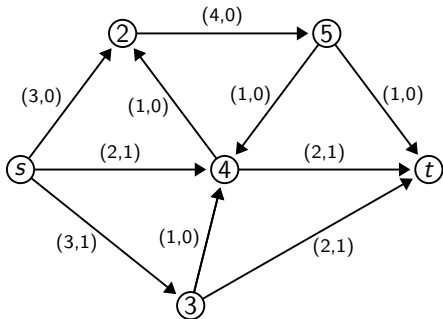
$R(G, x)$



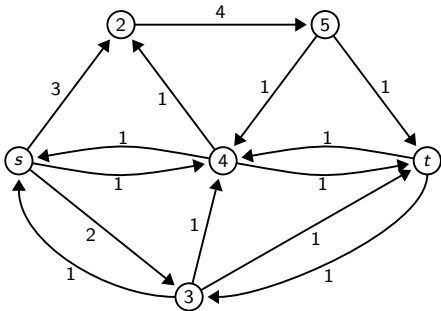


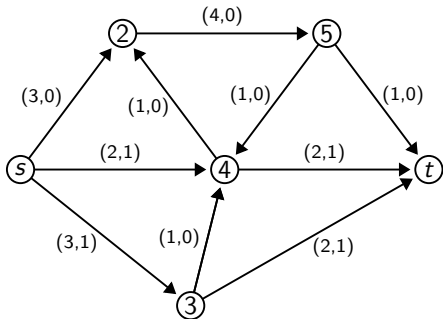
$R(G, x)$



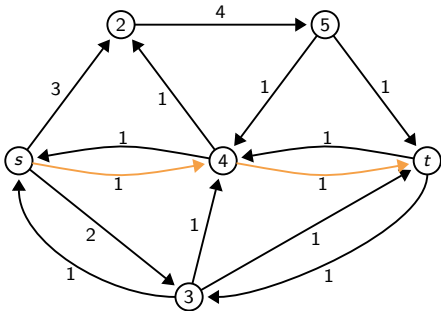


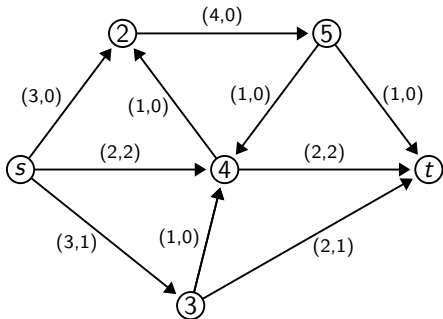
$R(G, x)$



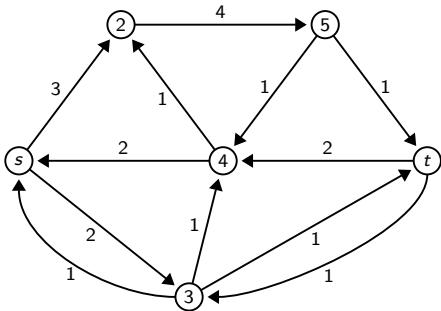


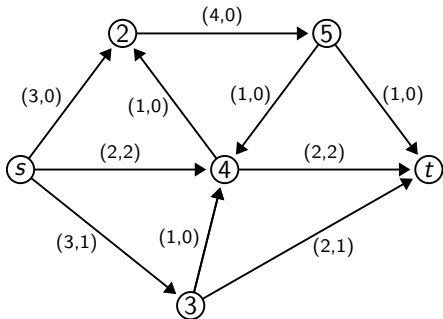
$R(G, x)$



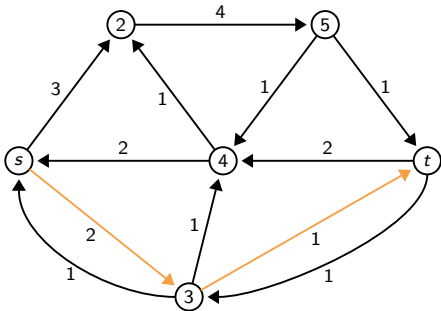


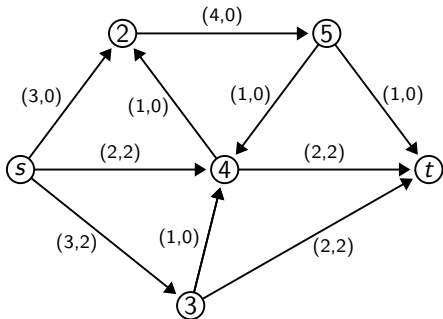
$R(G, x)$



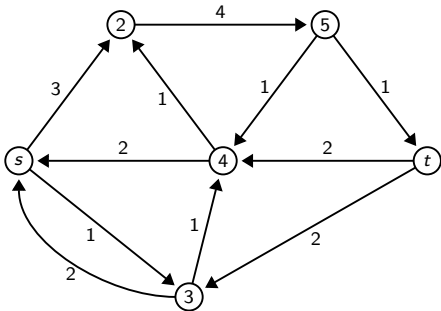


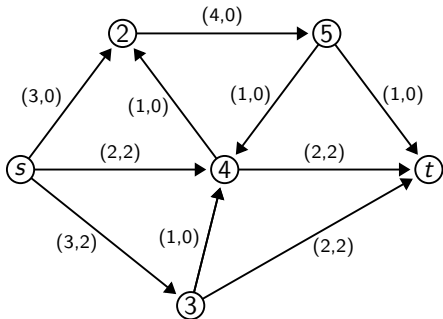
$R(G, x)$



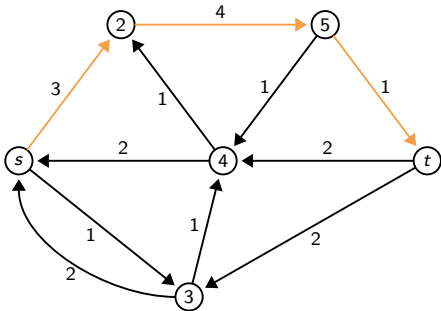


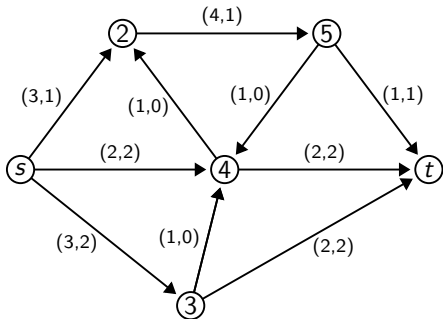
$R(G, x)$



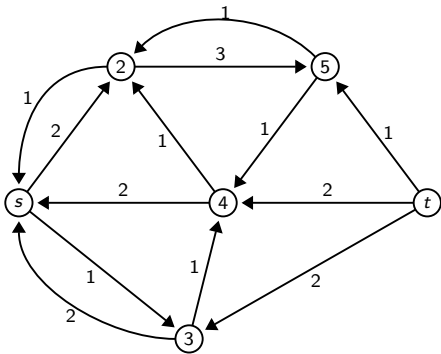


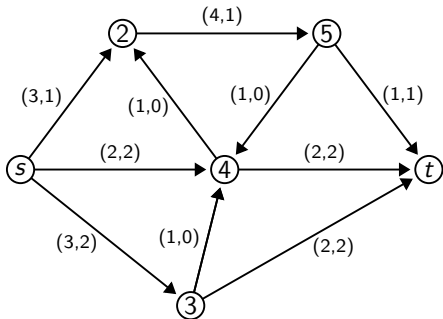
$R(G, x)$



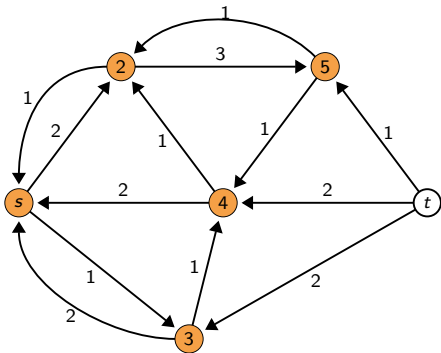


$R(G, x)$





$R(G, x)$



Algoritmo de Ford y Fulkerson

- ▶ **Teorema:** Si las capacidades de los arcos de la red son **enteras**, entonces el problema de flujo máximo tiene un flujo máximo entero.

Algoritmo de Ford y Fulkerson

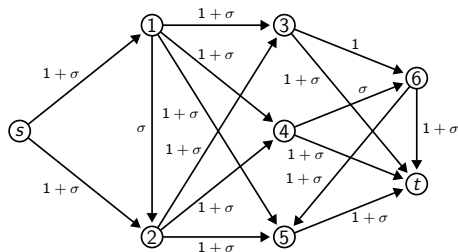
- ▶ **Teorema:** Si las capacidades de los arcos de la red son **enteras**, entonces el problema de flujo máximo tiene un flujo máximo entero.
- ▶ **Teorema:** Si los valores del flujo inicial y las capacidades de los arcos de la red son enteras, entonces el método de Ford y Fulkerson realiza a lo sumo nU iteraciones, donde U es una cota superior finita para el valor de las capacidades.

Algoritmo de Ford y Fulkerson

- ▶ **Teorema:** Si las capacidades de los arcos de la red son **enteras**, entonces el problema de flujo máximo tiene un flujo máximo entero.
- ▶ **Teorema:** Si los valores del flujo inicial y las capacidades de los arcos de la red son enteras, entonces el método de Ford y Fulkerson realiza a lo sumo nU iteraciones, donde U es una cota superior finita para el valor de las capacidades.
- ▶ Si las capacidades o el flujo inicial son **números irracionales**, el método de Ford y Fulkerson puede no parar (es decir, realizar un número infinito de pasos).

Algoritmo de Ford y Fulkerson

$$\sigma = (\sqrt{5} - 1)/2$$



Iteración	Camino de aumento
$6k + 1$	$s, 1, 2, 3, 6, t$
$6k + 2$	$s, 2, 1, 3, 6, 5, t$
$6k + 3$	$s, 1, 2, 4, 6, t$
$6k + 4$	$s, 2, 1, 4, 6, 3, t$
$6k + 5$	$s, 1, 2, 5, 6, t$
$6k + 6$	$s, 2, 1, 5, 6, 4, t$

Algoritmo de Edmonds y Karp



Jack Edmonds
(1934–)



Richard Karp
(1935–)

- ▶ La modificación de Edmonds y Karp (1972) a este algoritmo consiste en usar BFS para buscar caminos de aumento.
- ▶ Resuelve el problema con complejidad $O(nm^2)$.

Matching máximo en grafos bipartitos

- Un **matching o correspondencia** entre los vértices de G , es un conjunto $M \subseteq E$ de aristas de G tal que para todo $v \in V$, v es incidente a lo sumo a una arista de M .

Matching máximo en grafos bipartitos

- ▶ Un **matching o correspondencia** entre los vértices de G , es un conjunto $M \subseteq E$ de aristas de G tal que para todo $v \in V$, v es incidente a lo sumo a una arista de M .
- ▶ El problema de **matching máximo** consiste en encontrar un matching de cardinal máximo entre todos los matchings de G .

Matching máximo en grafos bipartitos

- ▶ Un **matching o correspondencia** entre los vértices de G , es un conjunto $M \subseteq E$ de aristas de G tal que para todo $v \in V$, v es incidente a lo sumo a una arista de M .
- ▶ El problema de **matching máximo** consiste en encontrar un matching de cardinal máximo entre todos los matchings de G .
- ▶ El problema de matching máximo es resoluble en tiempo polinomial para grafos en general (Edmonds, 1961–1965).

Matching máximo en grafos bipartitos

- ▶ Un **matching o correspondencia** entre los vértices de G , es un conjunto $M \subseteq E$ de aristas de G tal que para todo $v \in V$, v es incidente a lo sumo a una arista de M .
- ▶ El problema de **matching máximo** consiste en encontrar un matching de cardinal máximo entre todos los matchings de G .
- ▶ El problema de matching máximo es resoluble en tiempo polinomial para grafos en general (Edmonds, 1961–1965).
- ▶ Pero en el caso de grafo bipartitos, podemos enunciar un algoritmo más simple transformándolo en un problema de flujo máximo en una red.

Matching máximo en grafos bipartitos

Dado el grafo bipartito $G = (V_1 \cup V_2, E)$ definimos la siguiente red $N = (V', E')$:

Matching máximo en grafos bipartitos

Dado el grafo bipartito $G = (V_1 \cup V_2, E)$ definimos la siguiente red $N = (V', E')$:

- ▶ $V' = V_1 \cup V_2 \cup \{s, t\}$, con s y t dos vértices ficticios representando la fuente y el sumidero de la red.

Matching máximo en grafos bipartitos

Dado el grafo bipartito $G = (V_1 \cup V_2, E)$ definimos la siguiente red $N = (V', E')$:

- ▶ $V' = V_1 \cup V_2 \cup \{s, t\}$, con s y t dos vértices ficticios representando la fuente y el sumidero de la red.
- ▶ $E' = \{(i, j) : i \in V_1, j \in V_2, ij \in E\}$
 $\cup \{(s, i) : i \in V_1\}$
 $\cup \{(j, t) : j \in V_2\}$.

Matching máximo en grafos bipartitos

Dado el grafo bipartito $G = (V_1 \cup V_2, E)$ definimos la siguiente red $N = (V', E')$:

- ▶ $V' = V_1 \cup V_2 \cup \{s, t\}$, con s y t dos vértices ficticios representando la fuente y el sumidero de la red.
- ▶ $E' = \{(i, j) : i \in V_1, j \in V_2, ij \in E\}$
 $\cup \{(s, i) : i \in V_1\}$
 $\cup \{(j, t) : j \in V_2\}$.
- ▶ $u_{ij} = 1$ para todo $ij \in E$.

Matching máximo en grafos bipartitos

Dado el grafo bipartito $G = (V_1 \cup V_2, E)$ definimos la siguiente red $N = (V', E')$:

- ▶ $V' = V_1 \cup V_2 \cup \{s, t\}$, con s y t dos vértices ficticios representando la fuente y el sumidero de la red.
- ▶ $E' = \{(i, j) : i \in V_1, j \in V_2, ij \in E\}$
 $\cup \{(s, i) : i \in V_1\}$
 $\cup \{(j, t) : j \in V_2\}$.
- ▶ $u_{ij} = 1$ para todo $ij \in E$.

El cardinal del matching máximo de G será igual al valor del flujo máximo en la red N .